ТЕМА 3. КАСКАДНЫЕ ТАБЛИЦЫ СТИЛЕЙ

ЛЕКЦИЯ 3.21. CSS Flexible Box

Flex-контейнер. Оси. Направления и переносы. Выравнивание и распределение элементов. Изменение размеров

CSS модуль раскладки Flexible Box был разработан как модель одномерного-направленного макета и как один из методов распределения пространства между элементами в интерфейсе, с мощными возможностями выравнивания.

Flex контейнер

Чтобы использовать возможности flexbox, необходимо создать так называемый flex-контейнер. Для этого свойству **display** задаем значение **flex** или **inline-flex**.

После этого, все непосредственные потомки контейнера станут flex-элементами, а значит, приобретут определенный набор свойств.

В лекции рассмотрим все свойства по порядку, а в конце вернемся к списку свойств по умолчанию.

Оси flexbox

Одно из важнейших понятий flexbox – **оси**.

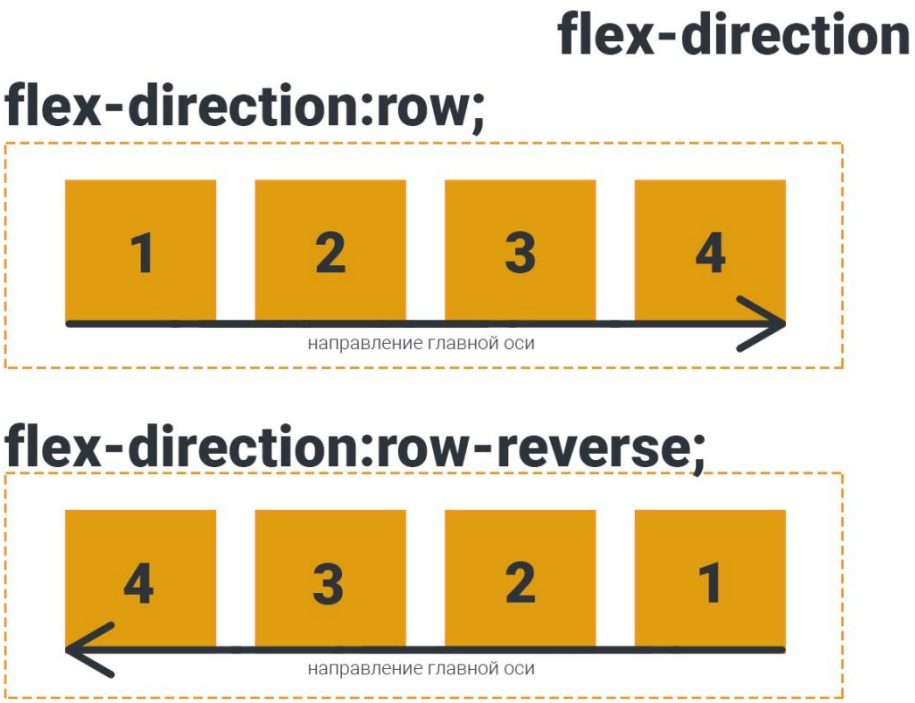
Их в модели существует две: главная и побочная.

Главную ось определяем с помощью свойства **flex-direction**, а побочная устанавливается автоматически перпендикулярно главной. Понимание того, с какой осью вы работаете (главная или побочная) очень важно для дальнейшего изучения Flexbox.

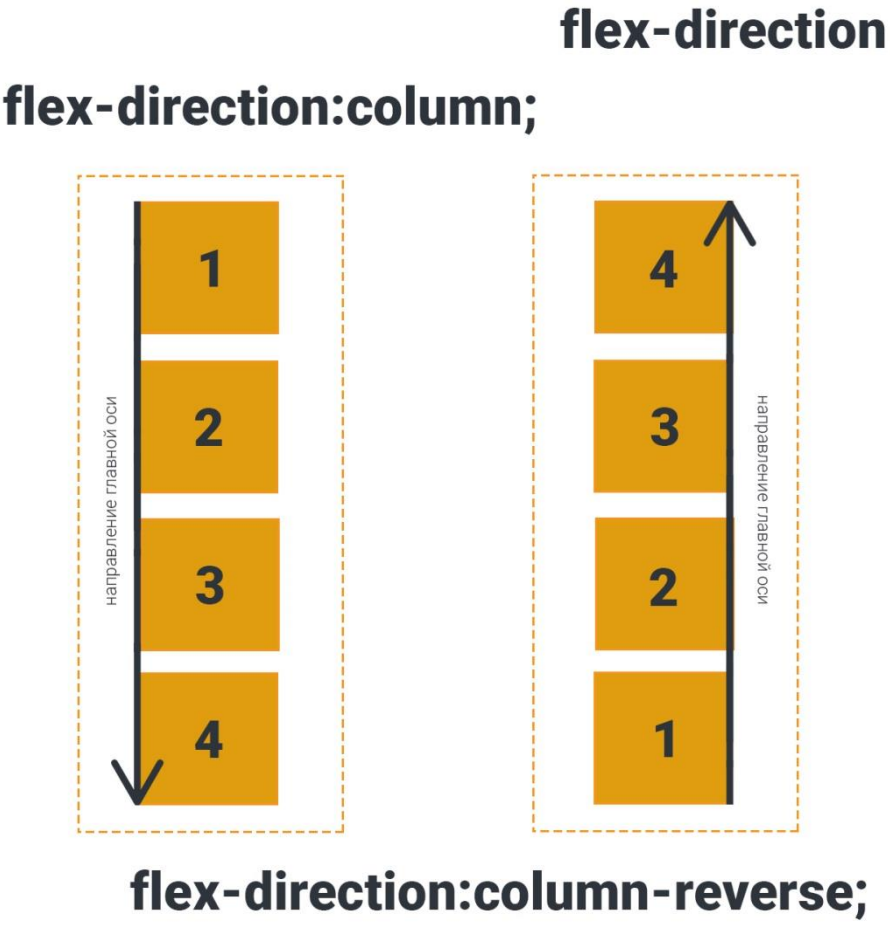
Для свойства доступны следующие значения:

Если вы установите **row** или **row-reverse**, главная ось будет проходить в горизонтальном направлении (inline direction).

©Pechora-PRO



Если установите **column** или **column-reverse**, то главная ось будет проходить в вертикальном направлении (block direction).



©Pechora-PRO

Начало и конец строки

Еще один важный момент с точки зрения верстки и ее адаптивности это направление текста: слева направо и справа налево.

Если свойству **flex-direction** задано значение **row** и вы работаете с английским языком, то начало главной оси будет слева, а конец главной оси – справа.

Если бы вы работаете с арабским языком, то начало главной оси будет справа, а конец главной оси – слева.

В обоих случаях начало побочной оси находится вверху flex контейнера, а конец – внизу, поскольку оба языка имеют режим горизонтальной записи.

Поэтому логичнее будет использовать термины начало и конец оси, а не левый и правый край.

Многострочные контейнеры. Flex-wrap

Так как flexbox – однонаправленная модель, то все элементы внутри гибкого блока будут стараться уместиться в одну строку. По умолчанию контейнерам внутри блока flex это разрешено. При этом установленные нами размеры блоков будут игнорироваться.

Если запретить им ужиматься, или если они не смогут ужаться достаточно сильно, то будет происходить переполнение контейнера.

Этот аспект поведения блоков задается с помощью свойства **flex-wrap**. По умолчание значение – **nowrap**.

Чтобы включить перенос блоков, изменим значение на **flex-wrap: wrap;** . Теперь, как только элементы будут слишком большими для того, чтобы помещаться на одну строку, они будут переноситься на новые.

©Pechora-PRO

Объединенное свойство. Flex-flow

Вы можете указать два свойства **flex-direction** и **flex-wrap** в одном **flex-flow**. Первое значение свойства определяет **flex-direction**, второе **flex-wrap**.

Полная запись	Эквивалент
flex-direction: row; flex-wrap: wrap;	**flex-flow:** row wrap;

Выравнивание и распределение элементов

Одной из причин огромной популярности и широкого использования flexbox является возможность выравнивать и распределять элементы по осям. Это в разы облегчает процесс расположения элементов на странице по сравнению с обычными css свойствами.

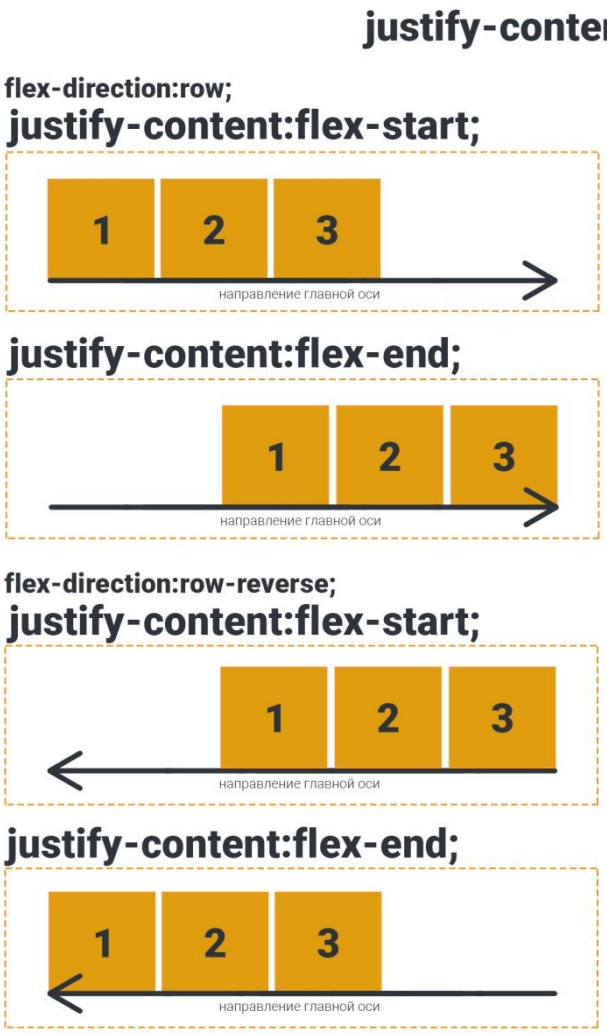
!!! Обратите внимание, что эти свойства должны быть установлены для гибкого контейнера, а не для самих элементов.

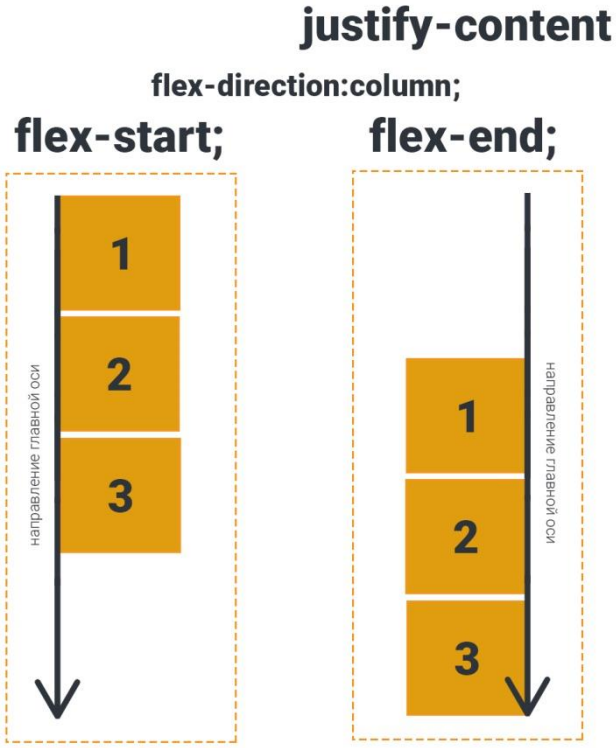
Свойство justify-content

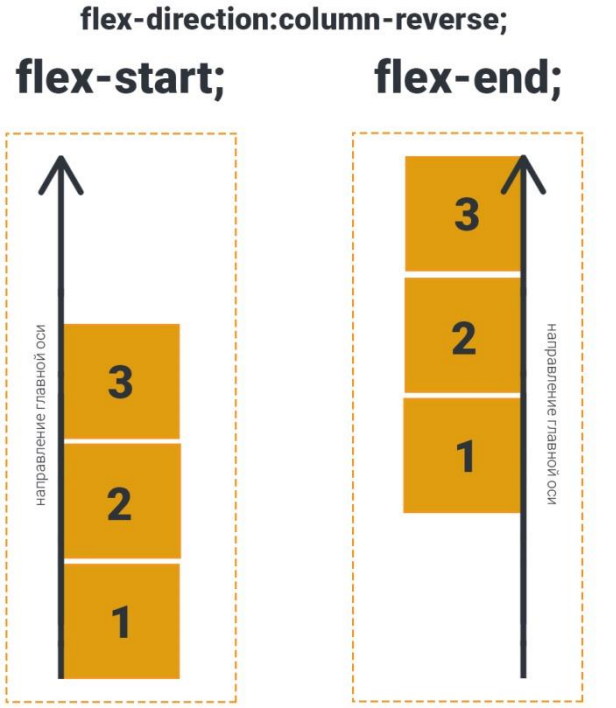
Свойство **justify-content** используется для выравнивания элементов по главной оси, направление которой задал **flex-direction**.

При выборе значений **flex-start** и **flex-end** расположение элементов будет зависеть от направления главной оси, в ее начале и конце соответственно.

©Pechora-PRO







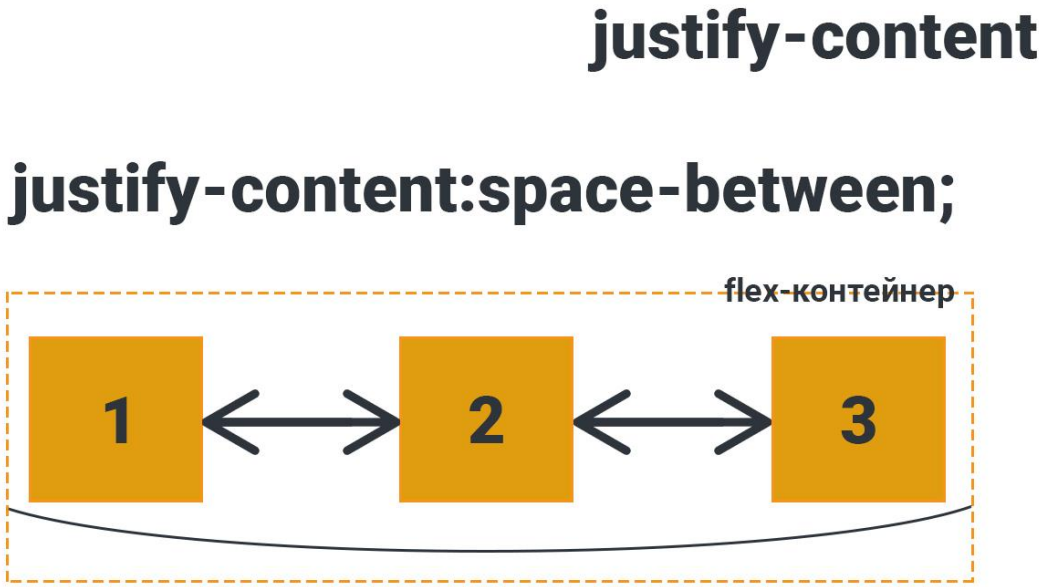
Значение **center** выровняет элементы по центру.

А вот следующие три свойства будут распределять элементы по строке со своими особыми настройками.

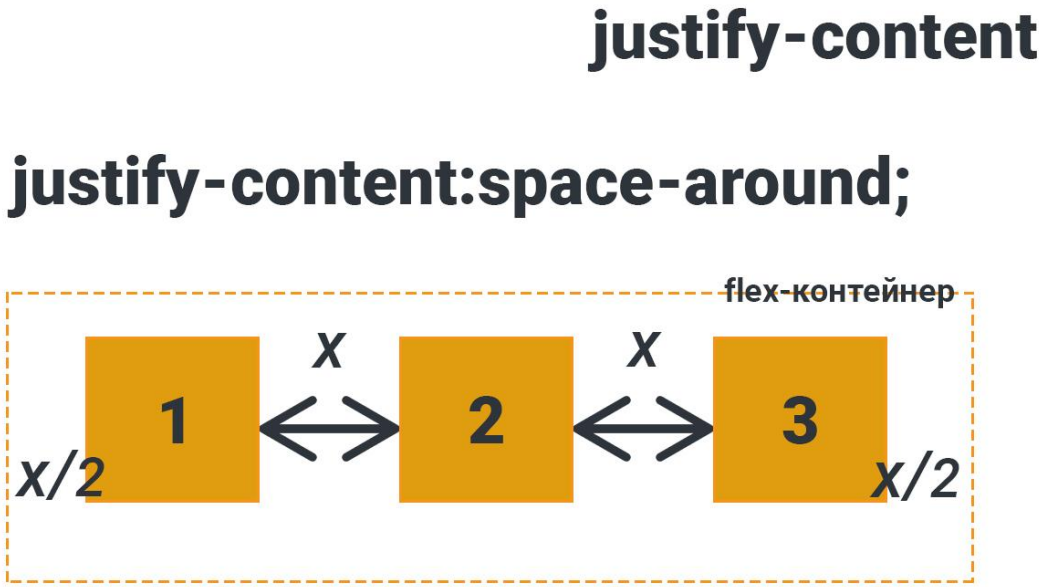
Space-between задаёт одинаковое расстояние между блоками, но не между контейнером и блоками.

©Pechora-PRO

Элементы занимают максимально доступную ширину внутри родителя.

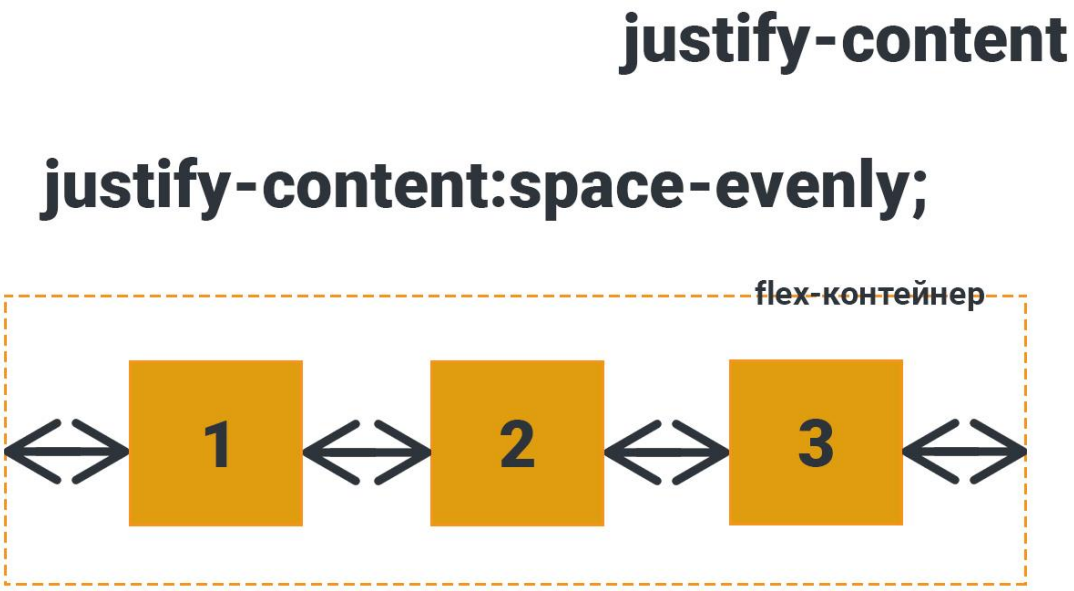


Space-around также задаёт одинаковое расстояние между блоками, но теперь расстояние между контейнером и блоками равно половине расстояния между блоками.



А чтобы элементы имели одинаковое пространство вокруг себя, используйте значение **space-evenly**.

©Pechora-PRO



Свойство align-items

Если **justify-content** работает с главной осью, то **align-items** работает с осью, перпендикулярной главной оси.

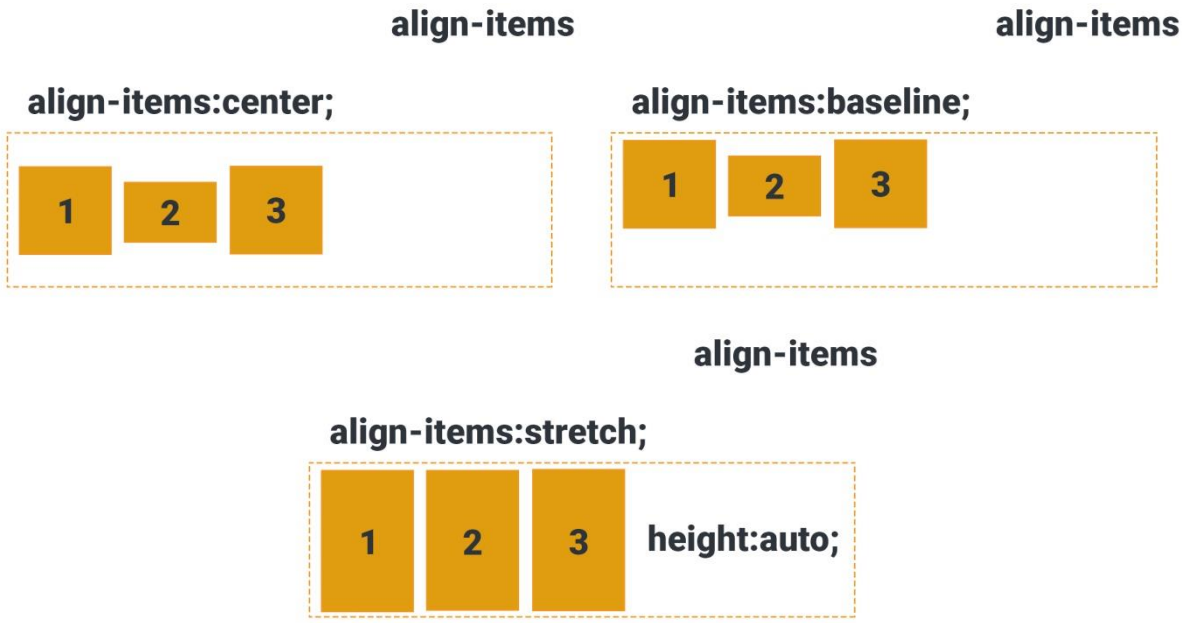
- flex-start
- flex-end
- center
- stretch – растягивает блоки по высоте родителя
- baseline – размещает по центру оси

Стоит заметить, что для **align-items: stretch** высота блоков должна быть равна auto.

На рисунках ниже иллюстрации применения свойства при **flex-direction:row;**

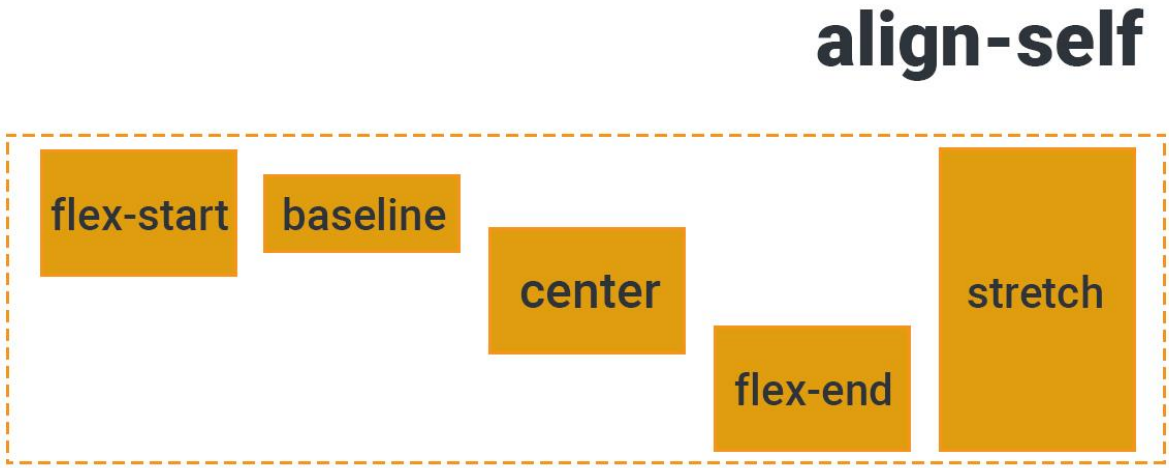


©Pechora-PRO



Свойство align-self

Если Вам необходимо задать разное расположение блокам внутри одного родителя, вы можете использовать свойство **align-self**. Значения такие же, как для **align-items**.



Установка размеров с flex-basis

Это свойство отвечает за изначальный размер элементов до того, как они будут изменены другими свойствами CSS Flexbox.

Свойство **flex-basis** определяет размер доступного пространства элемента.

©Pechora-PRO

Начальное значение этого свойства - **auto** - в этом случае браузер проверяет, имеют ли элементы размер.

Если элементы имеют ширину 100px, и этот размер читается браузером как flex-basis. Если элементы не имеют размера, то для размера flex-basis используется размер контента.

Вот почему, когда мы просто задаем display: flex родительскому элементу, все элементы перемещаются в ряд и занимают столько места, сколько им нужно для отображения их содержимого.

Изменение размеров

flex-grow

По умолчанию значение **flex-grow** равно 0. Это значит, что блокам **запрещено** увеличиваться в размерах.

Если задать **flex-grow** равным 1 для **каждого** блока, то они займут все оставшееся место в контейнере в равных пропорциях.

НО, **flex-grow** принимает не абсолютные значения, а **относительные**. Это значит, что не важно, какое значение у flex-grow, важно, какое оно по отношению к другим блокам.

На рисунке ниже, представлены 5 блоков, значение flex-grow каждого блока - 1, в сумме получится 5. Значит, каждый из блоков занимает 1/5 всего доступного пространства.

Когда **flex-grow** третьего блока становится равным 2, контейнер делится на 7 частей: 1 + 1 + 2 + 1 + 1. Теперь третий блок занимает 2/6 пространства, остальные — по 1/6. И так далее.

©Pechora-PRO



flex-shrink

Прямая противоположность **flex-grow**. Определяет, насколько блоку можно **уменьшиться** в размере. **Flex-shrink** используется, когда элементы не вмещаются в контейнер. Вы определяете, какие элементы должны уменьшиться в размерах, а какие — нет.

По умолчанию значение **flex-shrink** для каждого блока равно 1. Это значит, что блоки будут сжиматься, когда контейнер будет уменьшаться.



©Pechora-PRO

Если **flex-shrink** для блока равен 0, ему запретили сжиматься, поэтому его ширина останется изначальной.



flex-shrink основывается на пропорциях. То есть, если у первого блока **flex-shrink** равен 6, а у остальных он равен 2, то, это значит, что первый блок будет сжиматься в три раза быстрее, чем остальные.

Объединенное свойство. Flex

Заменяет **flex-grow**, **flex-shrink** и **flex-basis**.

Значения по умолчанию: 0 (**grow**) 1 (**shrink**) auto (**basis**).

Полная запись	Эквивалент
flex-grow: 2; **flex-shrink**: 3; **flex-basis**: auto;	**flex:** 2 3 auto;

©Pechora-PRO

Свойство order для визуального перемещения

CSS свойство **order** определяет порядок, используемый для размещения flex элементов в их flex контейнере. Элементы располагаются **в восходящем порядке** по значению order.

Значения задаются целыми числами, в том числе и отрицательными. Элементы с одинаковым значением **order** располагаются в том порядке, в каком они находятся в исходном коде.

Важно! Order воздействует **только** на **визуальный порядок** элементов, но не логический порядок или табуляцию.

Order не должен применяться к невизуальным медиаданным, таким как речь.



©Pechora-PRO

Значения свойств по умолчанию

Как и для всех свойств в CSS, некоторые начальные значения уже определены, поэтому при создании flex-контейнера все содержащиеся в нем flex-элементы будут вести себя следующим образом.

- Элементы отображаются в ряд (свойство flex-direction по умолчанию имеет значение row).
- Позиционирование элементов начинается от начала главной оси.
- Элементы не растягиваются по основной оси, но могут сжиматься.
- Элементы будут растягиваться, чтобы заполнить размер побочной оси.
- Свойству flex-basis задано значение auto.
- Свойству flex-wrap задано значение nowrap.

Результатом этого является то, что все ваши элементы будут выстроены в ряд, используя размер содержимого в качестве их размера на главной оси. Если в контейнере больше элементов, чем можно уместить, они не будут обёрнуты и контейнер будет переполнен. Если некоторые элементы выше других, все элементы будут вытянуты вдоль побочной оси, чтобы заполнить ось в полный размер.

©Pechora-PRO